```
#include <iostream>
#include <string>

using namespace std;

// Base class
class Person {
protected:
    string name;
    int age;
    string contact;

public:
    // Constructor using member initializer list and const references
    Person(const string& n, int a, const string& c)
        : name(n), age(a), contact(c) {}

    // Virtual destructor for base class
    virtual ~Person() = default;

    void displayPerson() const {
        cout << "Name: " << name << "\n";
        cout << "Age: " << age << "\n";
        cout << "Contact: " << contact << "\n";
    }
};

// Derived class: Employee (inherits from Person)
class Employee : public Person {
protected:
    int employeeID;
    string department;

public:
    // Delegating initialization to Person and using initializer list
    Employee(const string& n, int a, const string& c, int id, const string& dept)
        : Person(n, a, c), employeeID(id), department(dept) {}

    void displayEmployee() const {
        displayPerson();
        cout << "Employee ID: " << employeeID << "\n";
        cout << "Department: " << department << "\n";
    }
};
```

```cpp
// Derived class: Manager (inherits from Employee)
class Manager : public Employee {
private:
    int teamSize;

public:
    // Delegating initialization to Employee and using initializer list
    Manager(const string& n, int a, const string& c, int id, const string& dept, int size)
        : Employee(n, a, c, id, dept), teamSize(size) {}

    void displayManager() const {
        displayEmployee();
        cout << "Team Size: " << teamSize << "\n";
    }
};

int main() {
    Manager manager("Anjali", 40, "9876543210", 101, "HR", 8);

    cout << "--- Manager Details ---\n";
    manager.displayManager();

    return 0;
}
```

